UrbanFix AI — AI/ML Models, Algorithms & Security: Complete Flow-Based Explanation

Date: 2026-03-04
Purpose: Judge / Evaluator presentation document — explains every AI/ML model, algorithm, dataset, security approach, and edge-case handling in the UrbanFix AI platform, explained through the natural user flow.

Table of Contents

1. How the App Works — User Flow Overview
2. STEP 1 — User Opens App & Registers (Security Layer)
3. STEP 2 — User Sees the Feed (AI-Powered Ranking)
4. STEP 3 — User Creates an Issue Report (Where AI Begins)
5. MODEL 1 — Image Validation (Is This a Real Civic Issue?)
6. MODEL 2 — Object Detection & Classification (What Civic Issue Is This?)
7. MODEL 3 — Severity Assessment (How Bad Is This Issue?)
8. MODEL 4 — Duplicate Detection (Has Someone Already Reported This?)
9. MODEL 5 — Smart Priority Scoring (Which Issue Needs Attention First?)
10. MODEL 6 — NLP Text Analysis (Understanding What the User Wrote)
11. MODEL 7 — Department Routing (Where Should This Issue Go?)
12. MODEL 8 — Fraud / Fake Report Detection (Is This Report Genuine?)
13. STEP 4 — Admin Reviews & Assigns (AI-Assisted Triage)
14. STEP 5 — Field Worker Resolves (Resolution Verification Model)
15. MODEL 9 — Resolution Verification (Did the Worker Actually Fix It?)
16. STEP 6 — Analytics & Intelligence (Batch AI Models)
17. MODEL 10 — Civic Hotspot Detection
18. MODEL 11 — ETA Prediction
19. MODEL 12 — Resource Optimization
20. LLM (Large Language Model) Usage in UrbanFix AI
21. Complete AI Pipeline — End-to-End Flow Diagram
22. Datasets — What Data, Where From, How Cleaned
23. Data Cleaning & Preprocessing
24. Error Handling & Precision: How We Reduce Mistakes
25. Security Architecture — Complete Approach
26. Edge Case — Two Users Report Same Issue at Same Spot
27. Summary Table — All Models at a Glance
28. Evaluation Metrics — How We Measure Model Quality

---

1. How the App Works — User Flow Overview

Before diving into models, here is the complete flow. Every AI/ML model is marked with a 🤖 where it activates.

```
USER OPENS APP
    │
    ▼
REGISTER / LOGIN ──────────────────────── 🔒 Security: JWT + bcrypt + OAuth
    │
    ▼
LOCATION SETUP + PROFILE SETUP
    │
    ▼
HOME FEED ──────────────────────────────── 🤖 Priority Scoring Model (ranks posts)
    │                                      🤖 Fraud Detection (filters fakes)
    ├── Browse Issues (Community Feed)
    ├── Browse Municipal Updates
    ├── Upvote / Comment / Follow
    │
    ▼
USER TAPS "REPORT ISSUE"
    │
    ├── Selects Category
    ├── Writes Title + Description ──────── 🤖 NLP Text Analysis (extract
entities, sentiment, urgency)
    ├── Takes Photo / Video ─────────────── 🤖 Image Validation Model (is image
legitimate?)
    │                                      🤖 Object Detection - YOLO (what issue
is in the image?)
    │                                      🤖 Severity Assessment Model (how bad
is it?)
    ├── GPS Auto-Detected ───────────────── 🤖 Duplicate Detection Model (already
reported nearby?)
    ├── Anonymous / Emergency Toggles
    │
    ▼
ISSUE CREATED IN DATABASE
    │
    ├── 🤖 Smart Priority Score calculated
    ├── 🤖 Department Routing (auto-assign department)
    ├── 🤖 Fraud Score checked
    │
    ▼
ADMIN DASHBOARD (sees prioritized queue)
    │
    ├── Reviews AI-suggested severity, department, duplicates
    ├── Assigns field worker + deadline
    │
    ▼
FIELD WORKER DASHBOARD
    │
    ├── Sees assigned task with GPS directions
    ├── Goes to site → Fixes issue
    ├── Uploads "After" photo ──────────── 🤖 Resolution Verification Model
(before/after comparison)
    │
    ▼
```

```
ISSUE MARKED RESOLVED
    │
    ├── Citizen notified + points awarded
    ├── 🤖 Hotspot Detection (batch analytics)
    ├── 🤖 ETA model learns from resolution time
    └── 🤖 Resource Optimization (improve future routing)
```

---

2. STEP 1 — User Opens App & Registers (Security Layer)

What Happens

User creates account (email/password, OTP, or Google OAuth) and completes profile setup.

Security Approaches Used

Layer	Technology	Algorithm	Purpose
Password Storage	bcryptjs	Blowfish-based adaptive hashing (10 salt rounds)	Passwords are never stored in plain text. Even if database is breached, passwords cannot be reversed.
Session Token	JWT (JSON Web Token)	HMAC-SHA256 signing	After login, a signed token is issued (30-day validity). Every API request carries this token to prove identity.
OAuth (Google Login)	Supabase Auth + expo-auth-session	OAuth 2.0 / OpenID Connect	Delegates authentication to Google — the app never sees the user's Google password.
OTP Verification	Supabase Auth	Time-based random code (6 digits)	Email-based OTP adds a second verification factor.
Role-Based Access Control	Express middleware guards	Conditional `if (role === 'admin')` checks	Different users see different things — citizens cannot access admin dashboard, workers cannot assign tasks.

How JWT Works in This App

```
1. User logs in with email + password
2. Server verifies password hash (bcrypt.compare)
3. Server generates JWT: jwt.sign({ id, role }, SECRET, { expiresIn: '30d' })
4. Client stores JWT in AsyncStorage
5. Every API request attaches JWT: Authorization: Bearer <token>
6. Server middleware verifies JWT on each request
7. If valid → request proceeds with user identity
8. If invalid/expired → 401 Unauthorized
```

---

3. STEP 2 — User Sees the Feed (AI-Powered Ranking)

What Happens

The home feed shows reported issues ranked by relevance, not just chronological order.

AI Model Used: Smart Priority Scoring (see Model 5)

The feed order is determined by a **priority score** calculated for each issue. Issues with higher priority appear first. This means a critical pothole near a school appears above a minor graffiti issue, even if the graffiti was reported more recently.

---

4. STEP 3 — User Creates an Issue Report (Where AI Begins)

When a user taps "Report Issue", the following AI pipeline triggers **in sequence**:

```
User uploads photo
        │
        ▼
┌──────────────────┐
│ MODEL 1:         │  Is this a real outdoor civic issue photo?
│ Image Validation │  (Not a selfie, meme, blurry, or indoor photo)
└──────────────────┘
         │
         │ PASS ☑
         ▼
┌──────────────────┐
│ MODEL 2:         │  What type of issue is this?
│ Object Detection │  (Pothole? Garbage? Broken streetlight?)
│ (YOLOv8)         │  Also: WHERE in the image is the issue?
└──────────────────┘
         │
         ▼
┌──────────────────┐
│ MODEL 3:         │  How severe is this issue?
│ Severity Score   │  Combines image damage + location context
└──────────────────┘
         │
         ▼
┌──────────────────┐
│ MODEL 4:         │  Has someone already reported this same problem?
│ Duplicate Check  │  Compares location + image + text
└──────────────────┘
         │
         ▼
┌──────────────────┐
│ MODEL 6:         │  Understand the title/description
│ NLP Analysis     │  Extract entities, detect urgency, sentiment
└──────────────────┘
         │
         ▼
┌──────────────────┐
```

```
  │ MODEL 5:          │  Calculate final priority rank
  │ Priority Score    │  Combines all above + upvotes + age + location
  └───────────────────┘
          │
          │
          ▼
  ┌───────────────────┐
  │ MODEL 7:          │  Which department should handle this?
  │ Dept. Routing     │  Roads? Sanitation? Electricity?
  └───────────────────┘
          │
          │
          ▼
    Issue saved to database with all AI metadata
```

Now let's explain **each model in detail**.

---

5. MODEL 1 — Image Validation (Is This a Real Civic Issue?)

Purpose

Before running expensive AI detection, first verify the image is legitimate — not a selfie, screenshot, meme, blurry photo, or indoor picture.

Type of Model

Binary/Multi-class Image Classifier

Algorithm & Architecture

Aspect	Detail
Architecture	MobileNetV3-Large / EfficientNet-B0
ML Algorithm	Convolutional Neural Network (CNN) with transfer learning
Training Algorithm	Stochastic Gradient Descent (SGD) with momentum / AdamW optimizer
Loss Function	Binary Cross-Entropy (valid vs. invalid) or Categorical Cross-Entropy (multi-class rejection reasons)
Input	RGB Image resized to 224×224 pixels
Output	`isValid` (true/false) + rejection reason + confidence score

What It Checks (Validation Pipeline)

Check	Method	Threshold
Is image blurry?	Laplacian variance (OpenCV)	Variance < 100 → blurry → reject
Is it a screenshot?	Edge detection patterns + status bar detection	CNN classifier

Check	Method	Threshold
Is it an indoor photo?	Scene classification (outdoor vs indoor)	CNN classifier
Are faces visible?	Face detection (MTCNN / MediaPipe)	If faces found → auto-blur for privacy
GPS match?	EXIF GPS vs submitted GPS coordinates	Must be within 500m radius
Photo age?	EXIF timestamp	Must be taken within last 24 hours
Minimum resolution?	Pixel dimensions check	At least 640×480
File size?	Size in bytes	Minimum 50KB (reject placeholder images)

Why MobileNetV3?

- **Lightweight** — only 5.4M parameters (vs. ResNet-50 with 25M). Runs fast on CPU.
- **Pre-trained on ImageNet** — already understands visual features; we only fine-tune the last few layers.
- **Perfect for binary/simple classification** — we don't need a massive model to answer "is this a valid civic issue photo?"

Dataset

Dataset Source	Size	Purpose
Valid civic issue photos (from Kaggle pothole/garbage datasets)	10,000+	Positive class
Invalid photos (selfies, memes, indoor, screenshots, nature)	10,000+	Negative class
Web scraping (Google Images for indoor/outdoor scenes)	5,000+	Augmentation
Total	**25,000+**	Balanced training

How to Reduce Errors

1. **Data augmentation** — rotation, brightness changes, horizontal flip so the model doesn't overfit.
2. **Hard negative mining** — find images the model misclassifies, add them to training set.
3. **Confidence threshold** — only auto-reject if confidence > 90%. Between 70-90%, flag for manual review.
4. **Regular retraining** — as users submit more photos, add edge cases to the training data.

---

6. MODEL 2 — Object Detection & Classification (What Civic Issue Is This?)

Purpose

Detect, localize (draw bounding boxes), and classify civic issues in the user's uploaded photo. This is the **core AI model** of the entire application.

Type of Model

Object Detection

Algorithm & Architecture — YOLO (You Only Look Once)

Aspect	Detail
Architecture	YOLOv8 Medium (Ultralytics)
ML Category	Deep Learning — Convolutional Neural Network (CNN)
Core Algorithm	Single-shot detection: divides image into grid cells, each cell predicts bounding boxes + class probabilities simultaneously
Backbone	CSPDarknet53 (feature extraction)
Neck	PANet / FPN (multi-scale feature fusion)
Head	Decoupled head for classification + box regression
Training Algorithm	AdamW optimizer with cosine annealing learning rate schedule
Loss Functions	CIoU Loss (bounding box regression) + Binary Cross-Entropy (classification) + Distribution Focal Loss
Input	RGB Image resized to 640×640 pixels
Output	Bounding boxes + class labels + confidence scores

Why YOLO Over Simple CNN Classification?

Feature	Simple CNN Classifier	YOLO Object Detection
Can detect multiple issues in one photo?	❌ No — outputs only 1 label	☑ Yes — detects all issues
Shows WHERE the issue is in the image?	❌ No — just says "pothole"	☑ Yes — draws bounding box around pothole
Speed	Moderate	☑ Fastest (real-time capable)
Use case	"What is the main issue?"	"What issues are here, where are they, and how many?"

10 Detection Classes

Class ID	Issue Type	Visual Description
0	`pothole`	Hole/crack in road surface
1	`garbage`	Trash pile, scattered waste, overflowing bin
2	`streetlight_fault`	Broken/unlit/leaning streetlight

Class ID	Issue Type	Visual Description
3	`graffiti`	Unauthorized paint/markings on walls
4	`water_leak`	Water flowing from pipe/road/drain
5	`broken_sidewalk`	Damaged/cracked pedestrian path
6	`fallen_tree`	Tree fallen on road/sidewalk
7	`road_damage`	Large cracks, erosion, missing asphalt
8	`drainage_issue`	Blocked/overflowing drain, water logging
9	`encroachment`	Illegal construction/obstruction on public land

How YOLO Works — Step by Step

```
STEP 1: Input image (640×640) is divided into a grid (e.g., 80×80, 40×40, 20×20 at
3 scales)

STEP 2: Each grid cell predicts:
         - Is there an object centered in this cell? (objectness score)
         - If yes, what class? (10 probabilities — one per civic issue type)
         - Where exactly? (bounding box: x_center, y_center, width, height)

STEP 3: Many grid cells predict overlapping boxes. Non-Max Suppression (NMS)
removes
         redundant boxes, keeping only the most confident one per object.

STEP 4: Final output: list of detections, each with:
         - Class (e.g., "pothole")
         - Confidence (e.g., 0.92)
         - Bounding box coordinates (x1, y1, x2, y2)
```

Training Process

```python
from ultralytics import YOLO

# Step 1: Load YOLOv8 pre-trained on COCO dataset (80 general classes)
model = YOLO("yolov8m.pt")   # 'm' = medium model (25.9M params)

# Step 2: Fine-tune on our civic issue dataset
results = model.train(
    data="dataset.yaml",      # Our 10-class civic issue dataset
    epochs=100,               # Train for 100 passes through all data
    imgsz=640,                # Standard YOLO input resolution
    batch=16,                 # Process 16 images at a time
    lr0=0.01,                 # Initial learning rate
    lrf=0.001,                # Final learning rate (cosine decay)
    optimizer="AdamW",        # Optimizer algorithm
```

```
    augment=True,              # Enable data augmentation
    freeze=10,                 # Freeze first 10 layers (transfer learning)
    patience=15,               # Stop early if no improvement for 15 epochs
)

# Step 3: Export for deployment
model.export(format="onnx")  # Convert to ONNX for cross-platform serving
```

Why Transfer Learning?

Instead of training from scratch (which would need millions of images), we:

1. Start with YOLOv8 **pre-trained on COCO** (330,000 images, 80 classes like car, person, dog).
2. This model already "understands" edges, shapes, textures, objects.
3. We **freeze** the early layers (which detect generic features) and only **fine-tune** the later layers to learn civic-issue-specific features.
4. Result: We need only **5,000–10,000 images per class** instead of millions.

Dataset for Object Detection

Source	Images	Annotation Format	Notes
Kaggle Pothole Detection Dataset	5,000+	YOLO .txt (bounding boxes)	Pre-annotated with boxes around potholes
Roboflow Universe (search: "pothole", "garbage", "road damage")	10,000+	YOLO / COCO	Multiple community-contributed datasets
Custom Collection (UrbanFix app users)	5,000+	Annotated using Roboflow/CVAT	GPS-tagged real city photos
Google Open Images (filtered)	3,000+	COCO JSON → convert to YOLO	Filter for relevant classes
Synthetic augmentation (rotation, flip, mosaic, MixUp)	20,000+	Auto-generated	Increases training variety
Total	**~50,000**	YOLO .txt	70% train / 15% val / 15% test

YOLO Annotation Format

Each image has a `.txt` file with the same name. Each line = one detected object:

```
# Format: <class_id> <x_center> <y_center> <width> <height>
# All values normalized to 0-1 (relative to image dimensions)

0 0.45 0.60 0.30 0.25    ← Pothole at center-bottom of image
1 0.80 0.40 0.15 0.20    ← Garbage bin at right-center
```

How to Reduce Errors in YOLO

Technique	What It Does	Why It Helps
Data Augmentation	Randomly rotates (±15°), flips, changes brightness (±30%), adds noise	Prevents model from memorizing exact images — forces it to learn features
Mosaic Augmentation	Combines 4 training images into 1 compound image	Model sees multiple objects/contexts per training step
MixUp	Blends 2 images with interpolated labels	Smooths decision boundaries, reduces overconfidence
Early Stopping	Stops training when validation mAP stops improving (patience=15 epochs)	Prevents overfitting (memorizing training data instead of learning)
Confidence Threshold	Only accept detections with confidence ≥ 0.75	Reduces false positives (wrong detections)
Non-Max Suppression (NMS)	Removes duplicate/overlapping bounding boxes, keeps best one	Prevents 5 boxes on the same pothole
Cross-validation by city	Test set must include cities NOT in training set	Ensures model works in new locations, not just cities it was trained on
Regular retraining	Add misclassified images back to training set (hard negative mining)	Model learns from its own mistakes

Evaluation Metrics for YOLO

Metric	What It Measures	Target
mAP@0.5 (mean Average Precision at IoU 0.5)	Overall detection accuracy across all 10 classes — did the model find the right object in roughly the right location?	≥ 85%
mAP@0.5:0.95	Stricter accuracy — averaged across IoU thresholds 0.5 to 0.95	≥ 60%
Precision	Of all the detections the model made, how many were correct? (reduces false alarms)	≥ 90%
Recall	Of all the real issues in images, how many did the model find? (reduces misses)	≥ 85%
F1-Score	Harmonic mean of Precision and Recall (balanced measure)	≥ 87%
Inference Latency	Time per image on server	< 500ms
Per-class AP	Accuracy broken down per issue type — reveals weak classes	No class below 75%

Metric	What It Measures	Target
Confusion Matrix	Which classes get confused with each other (e.g., pothole vs road_damage)	Minimal off-diagonal values

7. MODEL 3 — Severity Assessment (How Bad Is This Issue?)

Purpose

After detecting WHAT the issue is, this model determines HOW SEVERE it is — from minor cosmetic damage to critical safety hazard.

Type of Model

Regression + Classification Hybrid (multi-head output)

Algorithm & Architecture

Aspect	Detail
Architecture	Dual-Input Network: CNN branch (image) + MLP branch (metadata), fused via concatenation
CNN Branch	ResNet-18 (processes the image) — extracts 512-dimensional visual feature vector
MLP Branch	3-layer Multi-Layer Perceptron (processes location, category, context as numbers) — outputs 64-dimensional context vector
Fusion	Concatenate 512 + 64 = 576-dimensional vector
Output Head 1	Single neuron with sigmoid activation → Severity Score (0–100)
Output Head 2	4-neuron softmax → Severity Level (Low / Medium / High / Critical)
Training Algorithm	AdamW optimizer, multi-task loss = MSE (score) + CrossEntropy (level)
Input Resolution	Image: 384×384, Metadata: feature vector of ~15 values

Architecture Diagram

```
┌──────────────────┐      ┌──────────────────────────────┐
│  Image Input     │      │  Metadata Input              │
│  (384×384 RGB)   │      │  [category, lat, lng, near_  │
│                  │      │   school, near_hospital,     │
│                  │      │   is_main_road, hour, season] │
└────────┬─────────┘      └──────────────┬───────────────┘
```

```
          │                              │
     ┌────▼────┐                   ┌────▼────┐
     │ ResNet-18│                   │  MLP    │
     │ CNN     │                   │ Dense(128)│
     │ (pre-   │                   │ Dense(64) │
     │  trained)│                   │ Dense(64) │
     └─────────┘                   └─────────┘
          │  512d                        │  64d
          └──────────────┬──────────────┘
                    ┌────▼────┐
                    │ Concat  │  576 dimensions
                    │ + Dense │
                    │  (256)  │
                    └─────────┘
                         │
              ┌──────────┴──────────┐
         ┌────▼────┐           ┌────▼────┐
         │ Score   │           │ Level   │
         │ (0-100) │           │ (4-cls) │
         │ Sigmoid │           │ Softmax │
         └─────────┘           └─────────┘
```

Severity Calculation — The Formula (Weighted)

Instead of using ONLY the model output, the final severity combines AI vision + real-world context:

```
FinalSeverityScore = (
    0.30 × VisualDamageScore    +    ← From CNN: how large/damaged the issue looks
    0.25 × CategoryBaseRisk     +    ← Streetlight=80, OpenManhole=95, Pothole=60,
Graffiti=20
    0.20 × LocationContext      +    ← Near school (+20), hospital (+25), highway
(+15)
    0.15 × HistoricalWeight     +    ← Repeat issue in area? Similar complaints
nearby?
    0.10 × SeasonalFactor            ← Monsoon (+15 for drainage), Winter (+5 for
road cracks)
)
```

Example Calculation:

```
Pothole near a school during monsoon season:

VisualDamageScore  = 70 (moderate-sized hole detected by CNN)
CategoryBaseRisk   = 60 (Pothole base score)
LocationContext    = 45 (School nearby = +20, Main road = +15, Dense area = +10)
HistoricalWeight   = 30 (3 similar reports in this area in last month)
SeasonalFactor     = 65 (Monsoon = +15 for road deterioration)

FinalScore = 0.30×70 + 0.25×60 + 0.20×45 + 0.15×30 + 0.10×65
```

```
            = 21 + 15 + 9 + 4.5 + 6.5
            = 56 → "Medium" heading toward "High"
```

Severity Level Mapping

Score Range	Level	Color	Response Time	Action
0 – 25	**Low**	🟢 Green	7–14 days	Standard queue
26 – 50	**Medium**	🟡 Yellow	3–7 days	Priority queue
51 – 75	**High**	🟠 Orange	24–72 hours	Immediate admin attention
76 – 100	**Critical**	🔴 Red	< 24 hours	Emergency escalation, auto-notify senior admin

Dataset for Severity Model

Source	Size	Labels
Annotated civic issue images with severity scores (expert-labeled by municipal engineers)	20,000+	Score (0–100) + Level (Low/Med/High/Critical)
Synthetic combinations (image + varying metadata)	30,000+	Generated with weighted formula

How to Reduce Errors

1. **Multi-task learning** — the score head and level head regularize each other.
2. **Expert labeling** — severity is subjective; use 3 municipal engineers per image and take the median.
3. **Feature importance analysis** — use SHAP/LIME to explain which factors drove the severity score.
4. **Calibration** — use temperature scaling to ensure confidence scores are meaningful.

---

8. MODEL 4 — Duplicate Detection (Has Someone Already Reported This?)

Purpose

When two users click a photo of the **same pothole at the same spot**, the system should detect this is a duplicate and merge reports instead of creating two separate issues.

Type of Model

Multi-modal Similarity Matching (Location + Image + Text combined)

How It Works — Step by Step

```
New issue submitted (image + GPS + title + category)
    │
    ▼
STEP 1: GEO-SPATIAL FILTER (PostGIS)
    │   Find all existing issues within 50m radius + same category
    │   Algorithm: Haversine distance formula via PostGIS ST_DWithin
    │   If no nearby issues found → NOT a duplicate → STOP
    │
    ▼
STEP 2: IMAGE SIMILARITY (CLIP Embeddings)
    │   Convert new image → 512-dimensional embedding vector (using OpenAI CLIP)
    │   Compare with embeddings of nearby issues (stored in pgvector/Milvus)
    │   Algorithm: Cosine Similarity
    │   If cosine similarity > 0.85 → potential duplicate
    │
    ▼
STEP 3: TEXT SIMILARITY (Sentence-BERT)
    │   Convert title + description → embedding vector (using all-MiniLM-L6-v2)
    │   Compare with text embeddings of nearby issues
    │   Algorithm: Cosine Similarity
    │   If text similarity > 0.80 → potential duplicate
    │
    ▼
STEP 4: COMBINED SCORE (Weighted Average)
    │   DuplicateScore = 0.40 × GeoProximity + 0.35 × ImageSimilarity + 0.25 ×
TextSimilarity
    │   If DuplicateScore > 0.75 → DUPLICATE DETECTED
    │
    ▼
STEP 5: ACTION
    ├── If duplicate → Show user: "This issue was already reported. Upvote
instead?"
    │                  Merge into existing issue, increment its upvote count
    └── If not duplicate → Create new issue normally
```

Algorithms Used

Component	Algorithm	Purpose
Geo-spatial filtering	Haversine formula / PostGIS `ST_DWithin`	Find issues within 50m radius
Image embedding	CLIP (Contrastive Language-Image Pre-training) by OpenAI	Convert image to 512-dimensional vector that captures semantic meaning
Text embedding	Sentence-BERT (`all-MiniLM-L6-v2` from sbert.net)	Convert text to embedding vector for semantic comparison
Similarity measurement	Cosine Similarity: $\cos(\theta) = \frac{A \cdot B}{\|A\| \|B\|}$	Measures how similar two vectors are (1.0 = identical, 0.0 = completely different)

Component	Algorithm	Purpose
Vector storage	pgvector (PostgreSQL extension) or Milvus	Store and retrieve high-dimensional embedding vectors efficiently

Why Cosine Similarity?

- **Scale-invariant** — magnitude doesn't matter, only direction. Two similar images have vectors pointing in similar directions.
- **Fast** — dot product + normalization, computable in microseconds.
- **Industry standard** — used by Google, OpenAI, and every major search engine for semantic similarity.

How the System Handles Two Users Reporting Same Spot

Scenario: User A reports a pothole at coordinates (19.2965, 72.8777). User B, 20 meters away, photographs the same pothole from a different angle.

```
User A submits first → Issue #101 created (no duplicates exist)
    │
    │   System stores:
    │   - GPS: (19.2965, 72.8777)
    │   - CLIP embedding of User A's photo → stored in vector DB
    │   - Sentence embedding of User A's title → stored in vector DB
    │
User B submits second →
    │
    ├── Step 1: PostGIS query: SELECT issues WHERE ST_DWithin(location,
B_location, 50m)
    │           Result: Issue #101 found (23m away, same category "pothole")
    │
    ├── Step 2: CLIP similarity between B's photo and A's photo = 0.88 (> 0.85
threshold)
    │           Even though angle is different, CLIP understands both show a
pothole
    │
    ├── Step 3: Text similarity ("Big pothole on road" vs "Dangerous hole in main
road") = 0.82
    │
    ├── Step 4: Combined = 0.40×0.95 + 0.35×0.88 + 0.25×0.82 = 0.38 + 0.31 + 0.21
= 0.90
    │           0.90 > 0.75 → DUPLICATE CONFIRMED
    │
    └── Step 5: System shows User B:
                "⚠ This issue was already reported by someone 23m away.
                 Would you like to upvote it instead?"
                [View Existing Issue]  [Report Anyway]

                If User B upvotes → Issue #101 gets +1 upvote + priority boost
                If User B reports anyway → Creates separate issue, but links them
as related
```

9. MODEL 5 — Smart Priority Scoring (Which Issue Needs Attention First?)

Purpose

Rank all reported issues so that admins see the most urgent ones first. A critical open manhole near a school should appear above a minor graffiti.

Type of Model

Gradient Boosting Regressor (XGBoost) + **Formula-based fallback**

Algorithm

Aspect	Detail
Primary Model	XGBoost (Extreme Gradient Boosting)
ML Category	Supervised Learning — Regression
Core Algorithm	Ensemble of decision trees, each tree corrects the errors of the previous one
Training Algorithm	Gradient descent on tree splits (splits chosen to minimize squared error)
Input	8-feature vector per issue
Output	Priority score (0–100)
Update Frequency	Re-calculated on every upvote, status change, or time decay

Priority Score Formula

```
PriorityScore = w₁×Severity + w₂×Upvotes + w₃×Age + w₄×Location + w₅×Emergency +
w₆×Credibility + w₇×Category + w₈×Duplicates

Where:
  w₁ = 0.25  │  Severity     = AI severity score (0-100)
  w₂ = 0.15  │  Upvotes      = log₂(upvote_count + 1) × 10     ← logarithmic to
prevent manipulation
  w₃ = 0.10  │  Age          = max(0, 100 - days_since_report × 2)  ← decays over
time
  w₄ = 0.15  │  Location     = Zone multiplier (Hospital=1.5, School=1.4,
Highway=1.3, Residential=1.0)
  w₅ = 0.15  │  Emergency    = 30 if emergency flag set, else 0
  w₆ = 0.05  │  Credibility  = Reporter's historical accuracy (0.5 to 1.5
multiplier)
  w₇ = 0.10  │  Category     = Base urgency (OpenManhole=30, StreetLight=25,
Pothole=15, Graffiti=5)
  w₈ = 0.05  │  Duplicates   = +5 per similar report (same location, same issue)
```

Why XGBoost?

- Handles **tabular data** (numbers like upvotes, age, coordinates) better than neural networks.
- **Fast** — inference in < 1 ms. Can re-rank the entire feed in milliseconds.
- **Interpretable** — feature importance shows which factors matter most.
- **No GPU needed** — runs on CPU, perfect for real-time re-ranking.

---

10. MODEL 6 — NLP Text Analysis (Understanding What the User Wrote)

Purpose

Analyze the title and description the user typed to extract key information, detect urgency, identify sentiment (frustrated/calm), and improve categorization.

Type of Model

Transformer-based NLP (BERT family)

Algorithm & Architecture

Aspect	Detail
Architecture	DistilBERT (Phase 1) → MuRIL (Phase 2 for Indian languages)
ML Category	Deep Learning — Transformer (self-attention mechanism)
Training Algorithm	Fine-tuning with AdamW + linear learning rate warmup
Loss Function	Token-level CrossEntropy (NER) + Sentence-level CrossEntropy (sentiment/urgency)
Input	Text (title + description, max 256 tokens)
Output	Entities, sentiment, urgency signals, language, spam score
Latency	< 500ms

What It Extracts

Task	Example Input	Output
Named Entity Recognition (NER)	"Big pothole near City Mall on MG Road"	Location: "City Mall, MG Road"
Urgency Detection	"This has been here for 3 weeks, children are at risk"	Urgency: HIGH, Duration: "3 weeks", At-risk: "children"
Sentiment Analysis	"Fed up with this garbage pile, nobody cares!"	Sentiment: Frustrated (-0.6), Anger indicators detected
Language Detection	"सड़क पर बड़ा गड्ढा है"	Language: Hindi
Spam Detection	"Buy cheap electronics!!! Click here"	Spam score: 0.95 → REJECT

Task	Example Input	Output
Category Refinement	"Streetlight not working since Monday"	Suggested category: "streetlight_fault" (supports YOLO detection)

Why BERT / DistilBERT?

- **Understands context** — "This light is not working" → knows "light" means streetlight in civic context, not general light.
- **DistilBERT is 60% faster** than full BERT with 97% of the accuracy.
- **MuRIL** (Multilingual Representations for Indian Languages) by Google supports English, Hindi, Marathi, Tamil, Telugu, Kannada, Bengali, Gujarati — critical for an Indian civic app.

Dataset

Source	Size	Format
Municipal complaint portal texts (311 data, Smart City portals)	50,000+	Text + entity annotations (IOB format)
In-app user submissions (with consent)	Growing	Raw text → annotated by team
Synthetic generation (GPT-generated civic complaints in multiple languages)	10,000+	Auto-labeled

---

11. MODEL 7 — Department Routing (Where Should This Issue Go?)

Purpose

Automatically assign the detected issue to the correct municipal department without admin intervention.

Type of Model

Multi-class Classifier + Rule Engine Fallback

Algorithm

Aspect	Detail
Primary	Random Forest / Logistic Regression (tabular features)
Fallback	Rule-based mapping table
Input	Category (from YOLO), location zone, description keywords, severity
Output	Department name + confidence
Departments	Roads, Sanitation, Electricity, Water, Parks, Security, Traffic, Building

Routing Rules (Rule Engine — Always Active as Fallback)

Detected Issue	Primary Department	Override Rules
Pothole / Road Damage	Roads	If on highway → NHAI division
Garbage / Drainage	Sanitation	If medical waste → Hazmat team
Streetlight Fault	Electricity	If pole physically damaged → Roads team
Water Leak	Water	If sewage related → Sanitation
Fallen Tree	Parks	If blocking road → Traffic + Parks
Open Manhole	Roads + Water	ALWAYS emergency escalation
Graffiti	Parks	If offensive content → Security
Encroachment	Building	If on road → Traffic

Why Not Only Rules?

Rules handle 80% of cases perfectly. But edge cases (streetlight leaning into road = Roads + Electricity?) need ML. The model learns from past admin corrections — when an admin manually changes department routing, that becomes training data for the model.

---

12. MODEL 8 — Fraud / Fake Report Detection (Is This Report Genuine?)

Purpose

Detect and flag fraudulent, spam, or gaming-the-system reports before they pollute the feed and waste admin time.

Type of Model

Anomaly Detection Ensemble

Algorithm

Aspect	Detail
Architecture	Isolation Forest + Rule-based signals
ML Category	Unsupervised/Semi-supervised Anomaly Detection
Input	User behavior patterns, report metadata, image metadata
Output	Fraud score (0 to 1) + specific flags

Fraud Signals Detected

Signal	How Detected	Weight
High report frequency	Same user > 50 reports/day	Critical

Signal	How Detected	Weight
Image reuse	Perceptual hash (pHash) comparison — same image uploaded multiple times	Critical
GPS spoofing	EXIF GPS ≠ submitted GPS (distance > 500m)	Critical
Copy-paste descriptions	Text hash matching across reports	High
New account + high activity	Account age < 24 hours + > 10 reports	Medium
Suspiciously fast resolution	Report → Resolved in < 5 minutes	Medium
VPN / unusual IP	GeoIP anomaly detection	Low

Why Isolation Forest?

- Designed specifically for **anomaly detection** — identifies outliers in the data.
- **Unsupervised** — doesn't need labeled "fraud" vs "genuine" data. It learns what "normal" looks like and flags anything unusual.
- **Fast** — can score each report in < 10ms.
- Works well with **small fraud datasets** (fraud is rare, so we can't train a balanced classifier).

---

13. STEP 4 — Admin Reviews & Assigns (AI-Assisted Triage)

What Happens

Admin opens the dashboard. Issues are already sorted by AI-calculated priority score. Each issue shows:

```
┌──────────────────────────────────────────────────────────┐
│ Issue #247: "Huge pothole on MG Road near school"        │
│                                                          │
│ 🤖 AI Detection:  Pothole (92% confidence)               │
│ 🤖 AI Severity:   HIGH (Score: 78/100)                   │
│ 🤖 Priority Rank: #3 in queue                            │
│ 🤖 Department:    Roads (auto-suggested)                 │
│ 🤖 Duplicates:    2 similar reports found (linked)       │
│ 🤖 Sentiment:     Frustrated citizen                     │
│ 🤖 Fraud Score:   0.02 (genuine)                         │
│                                                          │
│ [Acknowledge] [Assign Worker] [Change Department]        │
└──────────────────────────────────────────────────────────┘
```

The admin can accept AI suggestions or override them. **Every override becomes training data** for improving the models.

---

14. STEP 5 — Field Worker Resolves (Resolution Verification Model)

What Happens

Worker goes to site → fixes issue → takes "after" photo → uploads.

Now the AI must verify: **did the worker actually fix it, or just upload a random photo?**

---

15. MODEL 9 — Resolution Verification (Did the Worker Actually Fix It?)

Type of Model

Siamese Network (twin CNN with shared weights)

Algorithm & Architecture

Aspect	Detail
Architecture	Siamese Network: two identical ResNet-18 branches with shared weights
ML Category	Deep Learning — Metric Learning
Core Concept	Both images pass through the SAME CNN. The outputs are compared via a difference/distance layer. If the difference indicates "the issue is no longer visible", it's resolved.
Training Algorithm	Contrastive Loss: $L = (1-Y) \frac{1}{2} D^2 + Y \frac{1}{2} \max(0, m-D)^2$ where D = Euclidean distance, Y = label (0=same/unresolved, 1=different/resolved), m = margin
Input	Before image (640×640) + After image (640×640)
Output	`isResolved` (boolean) + confidence + change percentage

How It Works

```
BEFORE photo                    AFTER photo
(pothole visible)               (road repaired)
       │                               │
  ┌────▼────┐                     ┌────▼────┐
  │ ResNet-18│                    │ ResNet-18│    ← SAME weights (shared)
  │ (CNN)    │                    │ (CNN)    │
  └──────────┘                    └──────────┘
       │  512d                         │  512d
       └──────────────────────────────┘
                  │
              ┌───▼────┐
              │  Diff  │    Compute difference between the two embeddings
              │ Layer  │
              └────────┘
                  │
            ┌─────▼──────┐
            │  Classifier │    Is the issue resolved based on visual change?
            │  (FC + Sig) │
```

```
            └──────────────┘
                   │
                   │
            isResolved: true (94% confidence)
            changePercentage: 87%
```

Additional Verification Checks

Check	Method	Purpose
Same location?	GPS comparison (before vs after)	Ensure worker went to the actual site
Same scene?	Image registration (feature matching)	Ensure before/after are of the same spot
Sufficient time gap?	Timestamp difference	A fix uploaded 2 minutes after assignment is suspicious
Weather consistency?	Compare sky/lighting conditions	Cross-validate with weather API data

---

16. STEP 6 — Analytics & Intelligence (Batch AI Models)

After issues are resolved, batch analytics models run **periodically** (daily/weekly) to generate insights.

---

17. MODEL 10 — Civic Hotspot Detection

Purpose

Identify areas where issues cluster — helps municipal planners allocate budgets and prioritize infrastructure repair by zone.

Algorithm

Aspect	Detail
Primary Algorithm	DBSCAN (Density-Based Spatial Clustering of Applications with Noise)
ML Category	Unsupervised Learning — Spatial Clustering
Why DBSCAN?	Finds clusters of arbitrary shape (not just circles like K-Means). Naturally handles noise (isolated reports that aren't part of any cluster). Doesn't require specifying number of clusters in advance.
Parameters	`eps` = 200m (max distance between two points in a cluster), `min_samples` = 5 (minimum reports to form a hotspot)
Input	Historical issue data: (latitude, longitude, category, timestamp, severity)

Aspect	Detail
Output	Hotspot zones with centroid, radius, dominant issue type, trend (increasing/stable/decreasing)
Frequency	Runs daily as a batch job

How DBSCAN Works

```
1. Pick any unvisited point
2. Find all points within 200m radius (eps)
3. If ≥ 5 points found (min_samples) → start a cluster
4. Expand the cluster: for each neighbor, find ITS neighbors within 200m
5. Continue until no more points can be added
6. Mark isolated points (< 5 neighbors within 200m) as noise
7. Repeat for remaining unvisited points
8. Result: clusters of varying sizes and shapes, each representing a civic hotspot
```

Additional: Time Series Analysis

After identifying spatial clusters, use **ARIMA / Prophet** time series models to detect trends:

- "Pothole complaints in Zone 5 increased 40% month-over-month"
- "Drainage issues predicted to spike in May (pre-monsoon)"

---

18. MODEL 11 — ETA Prediction

Purpose

Predict how long an issue will take to resolve based on historical patterns.

Algorithm

Aspect	Detail
Model	XGBoost / LightGBM Regression
Input	Category, severity, department, location, current workload, historical resolution times
Output	Estimated hours to resolution + confidence interval
Latency	< 200ms

ETA Factors

Factor	Impact on ETA
Issue category	Streetlight (avg 2 days), Pothole (avg 7 days), Major repair (avg 30 days)
Severity level	Critical reduces ETA by 50%, Low increases by 100%

Factor	Impact on ETA
Department workload	High backlog → +40% ETA
Historical dept. performance	Average resolution time for this dept
Weather conditions	Monsoon → +50% for outdoor work
Budget cycle	End of fiscal year → -30% (rush to spend budget)

19. MODEL 12 — Resource Optimization

Purpose

Optimize daily routes and task assignments for field workers to maximize resolved issues per day.

Algorithm

Aspect	Detail
Model	Vehicle Routing Problem (VRP) solver + Reinforcement Learning
Algorithm	OR-Tools (Google's Operations Research library) for VRP, PPO (Proximal Policy Optimization) for RL
Input	Worker locations, pending task locations, task priorities, traffic conditions, worker skills
Output	Optimized daily route per worker + estimated completion count
Frequency	Once per shift (morning batch) + dynamic re-routing on emergency

20. LLM (Large Language Model) Usage in UrbanFix AI

Where LLMs Are Used

Use Case	LLM	How
Auto-generating issue titles	GPT-4 / Llama 3 (API)	If user provides description but no title, LLM generates a concise title: "Describe: The road near my house has a big hole since 2 weeks, very dangerous" → Title: "Dangerous pothole on residential road (2 weeks)"
Summarizing issue threads	GPT-4 / Llama 3	When an issue has 50+ comments, LLM generates a summary for admins: "Citizens report the pothole has worsened despite partial repair. 12 new complaints since last week."
Translating reports	GPT-4 / Llama 3 / MuRIL	Auto-translate Hindi/Marathi reports to English for admin dashboard and vice versa.

Use Case	LLM	How
Chatbot for citizen queries	Fine-tuned Llama 3 / GPT-4	"What's the status of the pothole I reported on MG Road?" → LLM queries the database and responds naturally.
Admin report generation	GPT-4	Generate weekly reports: "This week: 45 issues reported, 32 resolved. Top category: Potholes. Worst zone: Sector 12."
Smart notification text	GPT-4 / Llama 3	Instead of "Issue #247 status updated", generate: "Great news! The pothole you reported on MG Road has been repaired. Check the before/after photos!"

LLM Architecture for UrbanFix

```
User Query / Text
       │
       ▼
┌──────────────────┐
│  LLM Gateway     │     Routes to appropriate LLM
│  (API Router)    │     based on task type
└──────────────────┘
         │
         │
    ┌────┴──────────────────────┐
    │                           │
┌───▼──────┐            ┌───────▼──────┐
│ GPT-4    │            │ Llama 3      │
│ (via API)│            │ (self-host)  │
│ Complex  │            │ Simple       │
│ tasks    │            │ tasks        │
└──────────┘            └──────────────┘
```

Why Both GPT-4 and Llama 3?

- **GPT-4** (API): Best accuracy for complex tasks (summaries, reports), but costs $0.03/1K tokens.
- **Llama 3** (self-hosted): Free, good for simple tasks (title generation, translations), runs on own server. No data leaves the infrastructure.

---

21. Complete AI Pipeline — End-to-End Flow Diagram

```
USER UPLOADS PHOTO + WRITES DESCRIPTION + GPS AUTO-DETECTED
                 │
                 ▼
     ┌──────────────────────────────┐
     │  STAGE 1: VALIDATION         │
     │  Model: MobileNetV3          │
```

```
        │  Time: ~300ms              │
        │                            │
        │  ✓ Is outdoor photo?       │
        │  ✓ Is not blurry?          │
        │  ✓ Is not screenshot?      │
        │  ✓ GPS matches EXIF?       │
        │  ✓ Photo < 24 hours old?   │
        │                            │
        │  REJECT if invalid ──────────── "Please upload a clear outdoor photo"
        └────────────────────────────┘
                     │
                     │ PASS
                     ▼
        ┌────────────────────────────┐
        │  STAGE 2: DETECTION        │
        │  Model: YOLOv8             │
        │  Time: ~500ms              │
        │                            │
        │  Detect: Pothole (92%)     │
        │  BBox: [120,80,320,230]    │
        │  Tags: road, safety        │
        └────────────────────────────┘
                     │
                     │
                     ▼
        ┌────────────────────────────┐       ┌────────────────────────────┐
        │  STAGE 3: SEVERITY         │       │  STAGE 3b: NLP ANALYSIS    │
        │  Model: ResNet-18 + MLP    │       │  Model: DistilBERT         │
        │  Time: ~400ms              │       │  Time: ~500ms              │
        │                            │       │                            │
        │  Score: 78/100, HIGH       │       │  Entities: "MG Road"       │
        │  Factors: visual damage    │       │  Sentiment: Frustrated     │
        │   + near school + monsoon  │       │  Urgency: HIGH             │
        └────────────────────────────┘       └────────────────────────────┘
                     │                                  │
                     └──────────────────────────────────┘
                                   │
                                   ▼
        ┌────────────────────────────┐
        │  STAGE 4: DUPLICATE CHECK  │
        │  Models: CLIP + SBERT      │
        │  DB: pgvector              │
        │  Time: ~1000ms             │
        │                            │
        │  Geo: 23m from Issue #101  │
        │  Image sim: 0.88           │
        │  Text sim: 0.82            │
        │  Combined: 0.90 → DUP!     │
        │                            │
        │  → Merge with #101         │
        │     or show user prompt    │
        └────────────────────────────┘
                     │
                     │
                     ▼
        ┌────────────────────────────┐
        │  STAGE 5: PRIORITY + ROUTE │
        │  Models: XGBoost + Rules   │
```

```
      │  Time: ~100ms              │
      │                            │
      │  Priority: 82/100 (Rank#3)│
      │  Department: Roads         │
      │  Fraud score: 0.02 (OK)    │
      └────────────────────────────┘
                   │
                   │
                   ▼
          ISSUE SAVED TO DATABASE
          with all AI metadata attached

     Total pipeline time: ~2.5 seconds
```

---

22. Datasets — What Data, Where From, How Cleaned

Complete Dataset Requirements

Model	Dataset Type	Quantity Needed	Sources	Annotation Format
Object Detection (YOLO)	Images with bounding box annotations	50,000+ images (5K–10K per class)	Kaggle Pothole Detection, Roboflow Universe ("pothole", "garbage", "road damage"), Google Open Images, custom city collection, web scraping from municipal portals	YOLO `.txt` — one file per image: `<class_id> <x_center> <y_center> <w> <h>` (values 0–1)
Image Validation	Valid + invalid civic photos	25,000+ (balanced)	Valid: above YOLO dataset. Invalid: selfies, memes, indoor, screenshots from web scraping + ImageNet random classes	Binary labels (CSV: filename, is_valid)
Severity Assessment	Images + severity scores (0–100)	20,000+	Expert-labeled by municipal engineers (3 annotators per image, take median). Synthetic combos (image + metadata variations)	CSV: filename, severity_score, severity_level, category, location_factors
Duplicate Detection	Image pairs (similar/different)	30,000+ pairs	Synthetic pairs from same-location reports + intentionally augmented duplicates + truly different pairs	CSV: img1_path, img2_path, is_duplicate, similarity_score
NLP Text Analysis	Annotated complaint texts	50,000+ texts	Municipal complaint portals (311 data, Smart City APIs), user submissions (with consent), GPT-generated multilingual civic complaints	IOB format for NER, CSV for sentiment/urgency labels

Model	Dataset Type	Quantity Needed	Sources	Annotation Format
Resolution Verification	Before/After image pairs	10,000+ pairs	Collected from field worker submissions over time	CSV: before_path, after_path, is_resolved, change_percentage
Fraud Detection	Normal + anomalous behavior logs	100,000+ events	App usage logs, synthetic fraud patterns	Event logs with fraud flags
Hotspot Detection	Historical issue records with GPS	All app data	UrbanFix database export	PostgreSQL dump (lat, lng, category, time, status)
Priority / ETA	Historical issues with resolution times	All app data	UrbanFix database	Tabular CSV: features + target (priority/eta)

Key Public Datasets

Dataset	Source	Size	Pre-annotated?
Pothole Detection (COCO format)	Kaggle: `atulyakumar98/pothole-detection-dataset`	5,000+ images	☑ Yes, bounding boxes
Road Damage Detection 2022	IEEE Big Data Cup: `sekilab/RoadDamageDataset`	26,000+ images	☑ Yes, 4 damage classes
Garbage Classification	Kaggle: `mostafaabla/garbage-classification`	15,000+ images	☑ Yes, 12 waste classes
India Municipal Complaints	data.gov.in Smart City datasets	100,000+ text records	Partial (category labels)
COCO Dataset (pre-training)	cocodataset.org	330,000 images, 80 classes	☑ Yes (used for YOLO pre-training)
Open Images V7	Google	9M images, 600 classes	☑ Yes, filter for relevant ones
Roboflow Universe	roboflow.com/universe	Varies	☑ Mixed quality, community-contributed

23. Data Cleaning & Preprocessing

Image Data Cleaning Pipeline

```
RAW IMAGES COLLECTED
        │
        ▼
STEP 1: REMOVE CORRUPT FILES
        │  - Check file headers (valid JPEG/PNG/WebP magic bytes?)
        │  - Discard files < 10KB (likely broken)
        │  - Discard files that fail PIL.Image.open()
        │
        ▼
STEP 2: RESOLUTION FILTER
        │  - Minimum: 640×480 pixels
        │  - Resize to standard dimensions (640×640 for YOLO, 384×384 for
severity)
        │  - Maintain aspect ratio with padding (letterboxing)
        │
        ▼
STEP 3: DUPLICATE REMOVAL
        │  - Compute perceptual hash (pHash) for every image
        │  - Remove images with pHash hamming distance < 5 (near-identical images)
        │  - Keeps only one image per unique visual
        │
        ▼
STEP 4: ANNOTATION QUALITY CHECK
        │  - Verify bounding boxes are within image bounds
        │  - Remove annotations with area < 1% of image (too tiny to learn)
        │  - Remove annotations with area > 90% of image (too vague)
        │  - Cross-check: each image must have ≥1 annotation
        │
        ▼
STEP 5: CLASS BALANCE CHECK
        │  - Count images per class
        │  - If any class is > 3× the smallest class → undersample large classes
        │  - If any class has < 2,000 images → augment (see below)
        │
        ▼
STEP 6: PRIVACY SCRUB
        │  - Run face detector → blur all faces
        │  - Run license plate detector → blur all plates
        │  - Strip sensitive EXIF fields (keep GPS + DateTime only)
        │
        ▼
STEP 7: AUGMENTATION (for underrepresented classes)
        │  - Random rotation (±15°)
        │  - Horizontal flip (50% probability)
        │  - Brightness adjustment (±30%)
        │  - Mosaic augmentation (stitch 4 images into 1)
        │  - MixUp (blend 2 images with interpolated labels)
        │  - Random crop + resize
        │  - Color jitter (hue, saturation, contrast)
        │
        ▼
STEP 8: TRAIN / VAL / TEST SPLIT
        │  - 70% Training / 15% Validation / 15% Test
```

```
          │  - Stratified split (each class equally represented in all sets)
          │  - Geographic split: test set includes ≥2 cities NOT in training set
          │
          ▼
CLEAN DATASET READY FOR TRAINING
```

Text Data Cleaning (NLP Model)

```
RAW COMPLAINT TEXTS
        │
        ▼
STEP 1: Language detection (langdetect library)
STEP 2: Remove HTML tags, URLs, email addresses
STEP 3: Normalize Unicode (NFKD normalization)
STEP 4: Remove excessive punctuation (!!!! → !)
STEP 5: Fix common misspellings (domain-specific: "pothoal" → "pothole")
STEP 6: Tokenize using model's tokenizer (BERT WordPiece / SentencePiece)
STEP 7: Truncate to max 256 tokens
STEP 8: Annotate entities in IOB format (manually or semi-auto with spaCy pre-
labels)
```

---

24. Error Handling & Precision: How We Reduce Mistakes

Multi-Layer Error Prevention

Layer	Strategy	Details
1. Training	Cross-validation	5-fold CV for tabular models. Geographic stratification for image models (test on unseen cities).
2. Training	Early stopping	Stop training when validation metric stops improving (patience = 15 epochs). Prevents overfitting.
3. Training	Data augmentation	Random transforms prevent model from memorizing exact images.
4. Training	Class balancing	Undersample majority classes + oversample minority. Use focal loss to focus on hard examples.
5. Inference	Confidence thresholds	Only auto-accept predictions ≥ 75% confidence. 50–74% → manual review. < 50% → reject.
6. Inference	Model ensemble	Run 2 models (YOLO + EfficientNet classifier). Both must agree for auto-classification.
7. Inference	Fallback chain	If AI service is down → use rule-based defaults (current mock logic). Never crash the user experience.

Layer	Strategy	Details
8. Post-deploy	Human-in-the-loop	Admin can override any AI decision. Every override = training data for next model version.
9. Post-deploy	Monitoring & drift detection	Track live accuracy daily. If prediction distribution shifts → alert ML team → investigate.
10. Post-deploy	A/B testing	New model version serves 10% traffic. Compare metrics vs production. Only promote if better.

Confidence Tracking System

```
IF confidence ≥ 0.75:
    → Auto-classify, auto-route ☑
    → User sees: "AI detected: Pothole (92%)"

IF 0.50 ≤ confidence < 0.75:
    → Flag for admin review ⚠
    → User sees: "AI suggested: Road Damage (65%) — awaiting admin confirmation"
    → Admin reviews and corrects → correction becomes training data

IF confidence < 0.50:
    → Don't display AI prediction ❌
    → Use user's manual category selection
    → Log the image for later batch annotation

IF AI service is unreachable:
    → Use category-based severity defaults (current mock logic)
    → Log incident + alert engineering team
    → Auto-retry after 30 seconds
```

Precision vs. Recall Tradeoff (Explained Simply)

Metric	What It Means	In Our Context
Precision	Of everything the model said is a pothole, how many actually ARE potholes?	High precision = fewer false alarms. Admin trusts the system.
Recall	Of all ACTUAL potholes in photos, how many did the model correctly detect?	High recall = fewer missed issues. Citizens feel heard.
F1-Score	Harmonic mean of precision and recall. Balances both.	We target F1 ≥ 87% for production.

Our Priority: Precision first (reduce false alarms to build admin trust), then improve recall through more data.

---

25. Security Architecture — Complete Approach

Authentication & Authorization

Layer	Approach	Technology
Password security	Adaptive hashing (bcrypt, 10 rounds). Password is hashed before storage. Even if DB is leaked, passwords are irreversible.	bcryptjs
Session management	JWT tokens (HMAC-SHA256), 30-day expiry. Stateless — server doesn't store sessions.	jsonwebtoken
OAuth integration	Google Sign-In via Supabase Auth. App never sees Google password.	OAuth 2.0 / Supabase Auth
OTP verification	6-digit time-based code sent to email. Expires in minutes.	Supabase Auth
Role-based access control (RBAC)	Middleware guards: `protect` (any authenticated user), `admin` (admin + super_admin only), `fieldWorker` (worker + admin).	Express middleware
API protection	Every `/api/*` route requires valid JWT. Middleware normalizes `req.user` with id + role.	authMiddleware.js

Data Security

Threat	Protection	How
Data in transit	TLS 1.3 encryption	All HTTP traffic over HTTPS (Render + Supabase enforce TLS)
Data at rest	AES-256 encryption	Supabase encrypts storage at rest by default
SQL injection	Parameterized queries	Supabase JS client uses parameterized queries, not string concatenation
XSS / CSRF	Input validation + CORS + CSP	Express CORS middleware (currently wide-open, should be restricted in prod)
DDoS protection	Rate limiting	Should add `express-rate-limit` (e.g., 100 requests/min per IP)
File upload abuse	Multer restrictions	Only image/video MIME types, 50 MB max, random filenames
Sensitive logging	Password masking	Request logger masks `password` field in body logs

AI-Specific Security

Threat	Protection
Adversarial images	Image validation model rejects manipulated/synthetic images
Model poisoning	Admin corrections are reviewed before incorporation into training data

Threat	Protection
Data privacy (PII in images)	Auto-blur faces (MTCNN/MediaPipe) + license plates before storage & AI processing
Model theft	AI service runs in private subnet. Model weights in private S3 bucket with IAM. ONNX models are not exposed via public API.
Prompt injection (LLM)	System prompts are hardcoded. User input is sanitized before passing to LLM. LLM cannot execute database queries directly.

Privacy & Compliance

Regulation	How We Comply
Indian IT Act, 2000	Reasonable security practices (encryption, access control). Data breach notification procedures.
DPDP Act, 2023	Explicit consent for data collection. Purpose limitation (data used only for civic reporting). Data minimization. Right to erasure (delete account deletes all data).
Anonymous reporting	When anonymous toggle is ON, user_id is not linked to issue publicly. Only stored internally for abuse prevention.

---

26. Edge Case — Two Users Report Same Issue at Same Spot

This is THE most frequently asked question in civic apps. Here's exactly how our system handles it:

Scenario

User A and User B are both at the same intersection. Both see the same overflowing garbage bin. Both pull out their phones and report it.

Timeline

```
T+0 sec:  User A opens Report screen, takes photo
T+3 sec:  User A submits report

          → Image Validation: PASS ☑
          → YOLO Detection: "garbage" (89%)
          → Severity: Medium (45/100)
          → Duplicate Check: No nearby issues → This is a NEW issue
          → Issue #301 created
          → CLIP embedding of A's photo stored in vector DB
          → Text embedding stored in vector DB

T+30 sec: User B opens Report screen, takes photo (different angle)
T+35 sec: User B submits report

          → Image Validation: PASS ☑
          → YOLO Detection: "garbage" (91%)
```

```
          → Severity: Medium (48/100)
          → Duplicate Check:
               Step 1: PostGIS finds Issue #301 only 8m away, same category ☑
               Step 2: CLIP similarity of B's photo vs A's photo = 0.87 (> 0.85)
☑
               Step 3: Text similarity = 0.78 (< 0.80) — texts are slightly
different ❌
               Step 4: Combined score = 0.40×0.98 + 0.35×0.87 + 0.25×0.78 = 0.89
                       0.89 > 0.75 → DUPLICATE DETECTED ☑

          → User B sees prompt:
            ┌──────────────────────────────────────────────┐
            │  ⚠️ Similar issue already reported!           │
            │                                              │
            │  "Garbage overflow at Main Road junction"    │
            │  Reported 30 seconds ago • 8m away           │
            │  Status: Submitted • 1 upvote                │
            │                                              │
            │  [👍 Upvote This Issue]  [📝 Report Separately]  │
            └──────────────────────────────────────────────┘

          → If User B taps "Upvote": Issue #301 gets +1 upvote, priority
recalculated
          → If User B taps "Report Separately": Issue #302 created, linked to #301
as related

T+5 min:  User C reports same garbage (3rd person)
          → Duplicate detection catches it again
          → Issue #301 now has 3 upvotes from related reports
          → Priority score increases: +5 per duplicate boost
          → Admin sees one consolidated issue with high community engagement
```

What If They Report 200m Apart (Not Exact Same Spot)?

- PostGIS **geo-filter uses 50m radius** by default.
- If 200m apart → geo-filter does not find Issue #301 → User B's report is treated as NEW
- However, **DBSCAN hotspot detection** (daily batch job) will later identify both as part of the same cluster
- Admin dashboard shows the area as a "hotspot" with 2 reports

What If GPS is Slightly Off?

- Our 50m radius accounts for GPS drift (typical smartphone GPS accuracy is 5–15m).
- CLIP image similarity acts as a backup — even if GPS is 40m off, if the photos look the same, duplicate is detected.
- That's why we use **multi-modal matching** (location + image + text combined) — no single signal is relied upon alone.

---

27. Summary Table — All Models at a Glance

#	Model	Algorithm	Input	Output	Latency	Purpose
1	**Image Validation**	CNN (MobileNetV3), transfer learning	Photo (224×224)	Valid/Invalid + reason	< 300ms	Reject bad/fake photos before AI pipeline
2	**Object Detection**	YOLOv8 (CSPDarknet + PANet + decoupled head)	Photo (640×640)	Bounding boxes + classes + confidence	< 500ms	Detect and classify civic issues in images
3	**Severity Assessment**	Dual-input CNN (ResNet-18) + MLP, multi-head output	Photo (384×384) + metadata	Score (0–100) + Level (L/M/H/C)	< 400ms	Grade issue severity for triage
4	**Duplicate Detection**	CLIP + Sentence-BERT + PostGIS, cosine similarity	Photo + GPS + text	isDuplicate + matched issues	< 1000ms	Merge same-spot reports
5	**Priority Scoring**	XGBoost gradient boosting regressor	8-feature vector	Score (0–100)	< 100ms	Rank issues for admin queue
6	**NLP Text Analysis**	DistilBERT / MuRIL (transformer, self-attention)	Text (256 tokens)	Entities + sentiment + urgency + language	< 500ms	Understand citizen descriptions
7	**Dept. Routing**	Random Forest classifier + rule engine fallback	Category + location + text	Department + confidence	< 200ms	Auto-assign to correct department
8	**Fraud Detection**	Isolation Forest (anomaly detection) + rules	User behavior + metadata	Fraud score (0–1) + flags	< 500ms	Catch fake/spam reports
9	**Resolution Verify**	Siamese Network (twin ResNet-18, contrastive loss)	Before photo + After photo	isResolved + change %	< 800ms	Verify worker actually fixed the issue
10	**Hotspot Detection**	DBSCAN spatial clustering + ARIMA time series	Historical GPS data	Hotspot zones + trends	< 5s (batch)	Identify problem areas for planning

#	Model	Algorithm	Input	Output	Latency	Purpose
11	**ETA Prediction**	XGBoost / LightGBM regression	Issue features + dept workload	Hours to resolution	< 200ms	Set resolution expectations
12	**Resource Optimization**	VRP (OR-Tools) + Reinforcement Learning (PPO)	Worker/task locations + priorities	Optimized daily routes	< 10s (batch)	Maximize issues resolved per day
LLM	**Language Intelligence**	GPT-4 (API) + Llama 3 (self-hosted)	Text queries	Natural language responses	1–5s	Title gen, summaries, translations, chatbot

28. Evaluation Metrics — How We Measure Model Quality

Per-Model Metrics & Minimum Targets

Model	Primary Metric	Target	Secondary Metrics
Object Detection (YOLO)	mAP@0.5	≥ 85%	mAP@0.5:0.95 ≥ 60%, Per-class AP ≥ 75%
Image Validation	F1-Score	≥ 90%	Precision ≥ 92%, Recall ≥ 88%
Severity Assessment	MAE (Mean Absolute Error)	≤ 10 points	Level Classification Accuracy ≥ 85%
Duplicate Detection	Precision@K	≥ 95%	False Positive Rate < 3%
Priority Scoring	Spearman Rank Correlation	ρ ≥ 0.85	RMSE, Kendall's Tau
Dept. Routing	Top-1 Accuracy	≥ 90%	Top-2 Accuracy ≥ 98%
Resolution Verification	Accuracy	≥ 90%	Cohen's Kappa ≥ 0.80
NLP Text Analysis	Entity F1 (NER)	≥ 80%	Sentiment F1 ≥ 82%
Fraud Detection	Precision	≥ 98%	Recall ≥ 70% (minimize false accusations)
Hotspot Detection	Silhouette Score	≥ 0.6	Cluster purity ≥ 80%
ETA Prediction	MAE (hours)	≤ 6 hours	R^2 ≥ 0.75

Model Improvement Cycle

```
Deploy v1.0 → Collect real predictions → Compare with admin corrections
        │                                          │
        ▼                                          ▼
    Monitor metrics                      Build correction dataset
    (accuracy, latency, drift)           (ground truth from admin overrides)
        │                                          │
        ▼                                          ▼
    If metrics drop → trigger             Retrain model v2.0
    retraining pipeline                   with new data
        │                                          │
        └──────────── Canary deploy (10% traffic) ─┘
                      Compare v2.0 vs v1.0
                      If better → promote to 100%
                      If worse  → rollback
```

Quick Reference — Technologies per AI Function

Function	ML Framework	Serving	Database
Object Detection	Ultralytics (PyTorch)	FastAPI + ONNX Runtime	—
Image Processing	OpenCV, Pillow	FastAPI	—
Embeddings (CLIP)	OpenAI CLIP (PyTorch)	FastAPI	pgvector / Milvus
Text Embeddings	Sentence-BERT (HuggingFace)	FastAPI	pgvector
NLP Models	HuggingFace Transformers	FastAPI	—
Tabular ML	XGBoost / scikit-learn	FastAPI	PostgreSQL
Spatial Clustering	scikit-learn (DBSCAN)	Batch job (cron)	PostgreSQL + PostGIS
LLM	OpenAI API / vLLM (Llama 3)	API gateway	Redis (cache)
VRP Optimization	Google OR-Tools	Batch job (daily)	PostgreSQL
Monitoring	Evidently AI, Prometheus	Grafana	—
Experiment Tracking	MLflow	Self-hosted	PostgreSQL
Dataset Versioning	DVC	S3 remote storage	Git

End of Document
Created: 2026-03-04
Purpose: College presentation to judges — comprehensive AI/ML, algorithm, dataset, security, and edge-case explanation for UrbanFix AI.